coursera | Vanderbilt University

Claude Code: Software Engineering with Generative AI... Agents

Module 1
Scaling Up Software Engineering with Claude Code & Generative AI

Module 2
Generative AI, Claude Code, & Code Quality
- Can AI Judge Code Quality? Video • 8 min
- Exercise: AI Evaluation of Code & Feature Implementations Reading • 10 min
- Does AI Labor Understand Design Principles? Video • 4 min
- Chat, Craft, Scale: Spend More Time Designing & Innovating Video • 2 min
- Chat: Craft and Explore Requirements & Options Video • 4 min
- Chat: Rapid Prototyping & Personas Video • 7 min
- Craft: Constraints & Prompts for Claude Code Video • 9 min

Module 3
Building Process & Context in Claude Code
- Global Persistent Context: CLAUDE.md Video • 10 min
- Writing Claude.md Files Reading • 10 min
- Reusable Targeted Context & Process: Claude Code Commands Video • 8 min
- Creating Claude Commands Reading • 10 min
- In-Context Learning: Teaching with Examples Video • 7 min
- Exercise: Building a Documentation Generator Reading • 10 min
- Contribute: Help Build a Curated Resource of Great Commands, Claude.md, etc. Reading • 10 min

Module 4
Version Control & Parallel Development with Claude Code

Module 5
Improving Claude Code Scalability & Reasoning

Module 6
Multimodal Prompting & Process

Writing Claude.md Files

What is Claude.md?

Claude.md is your AI team member's persistent memory - essential context that's available in every prompt. Just like onboarding a new developer, you need to provide Claude Code with the institutional knowledge it needs to work effectively on your team.

The CONTEXT Framework for Claude.md Design

Use this acronym to remember the key principles:

- **C**lear and Concise Instructions
- **O**perational Processes
- **N**aming and Standards
- **T**esting and Quality Gates
- **E**xamples and References
- **X**pectations and Boundaries
- **T**ools and Dependencies

Core Principles

1. Essential Information Only

Claude.md should contain **global** information that applies to every task. Don't overwhelm it with project-specific details that only apply to certain features.

2. Specificity Creates Better Targets

Context narrows the target from broad instructions to precise outcomes. "Write solid code" is vague; "Follow SOLID design principles for all object-oriented code" is actionable.

3. Process Over Micromanagement

Define workflows and checks, not implementation details. Let Claude make architectural decisions within your guardrails.

Example 1: Web Application Development Team

```
# Project: TaskFlow Web Application

## Core Principles
**IMPORTANT**: Whenever you write code, it MUST follow SOLID design principles. Never write code that violates these principles. If you do, you will be asked to refactor it.

## Development Workflow
1. Before making any changes, create and checkout a feature branch named `feature-[brief-description]`
2. Write comprehensive tests for all new functionality
3. Compile code and run all tests before committing
4. Write detailed commit messages explaining the changes and rationale
5. Commit all changes to the feature branch

## Architecture Overview
- **Frontend**: Next.js 14 with TypeScript and Tailwind CSS
- **State Management**: Zustand for client state, React Query for server state
- **Backend**: Node.js with Express and Prisma ORM
- **Database**: PostgreSQL
- **Testing**: Jest for unit tests, Playwright for E2E

## Code Standards
- Use TypeScript for all new code with strict type checking
- Follow the existing component structure in `/src/components`
- API routes follow RESTful conventions in `/src/pages/api`
- Use Prisma schema definitions for all database operations
- CSS classes should use Tailwind utilities; custom CSS only when necessary

## Quality Gates
- All code must compile without warnings
- Test coverage must remain above 80%
- All tests must pass before committing
- ESLint and Prettier must pass without errors

## File Organization
- Components: `/src/components/[feature]/[ComponentName].tsx`
- Pages: `/src/pages/[route].tsx`
- Utilities: `/src/lib/[category]/[utility].ts`
- Types: `/src/types/[domain].ts`
```

Example 2: Python Data Science Project

```
# Project: Customer Analytics Pipeline

## Development Standards
- **Language**: Python 3.11+
- **Code Style**: Follow PEP 8 strictly, use Black for formatting
- **Type Hints**: Required for all function signatures and class definitions
- **Documentation**: Docstrings required for all public functions and classes

## Workflow Requirements
1. Create feature branch: `analysis-[description]` or `model-[description]`
2. Write unit tests for all data processing functions
3. Run `pytest` and ensure all tests pass
4. Run `black .` and `flake8` before committing
5. Update relevant documentation in `/docs` if adding new features

## Project Structure
- `/src/data`: Data ingestion and preprocessing modules
- `/src/models`: ML model definitions and training scripts
- `/src/analysis`: Exploratory analysis notebooks and scripts
- `/src/utils`: Shared utility functions
- `/tests`: Comprehensive test suite
- `/configs`: Configuration files for different environments

## Data Handling Standards
- Use Pandas for data manipulation, prefer vectorized operations
- All data files must be documented in `/data/README.md`
- Use Pydantic models for data validation and serialization
- Never commit raw data files to version control
- Use environment variables for database connections and API keys

## ML/Analysis Guidelines
- Use scikit-learn for standard ML algorithms
- Notebook naming: `YYYY-MM-DD-[initials]-[description].ipynb`
- Save all trained models with versioning in `/models/trained`
- Use MLflow for experiment tracking
- Include model performance metrics in commit messages

## Dependencies
- Core: pandas, numpy, scikit-learn, matplotlib, seaborn
- ML: xgboost, lightgbm, optuna
```

Example 3: Mobile App Development (React Native)

```
# Project: FitnessTracker Mobile App

## Platform Requirements
- **Framework**: React Native 0.72+ with TypeScript
- **State Management**: Redux Toolkit with RTK Query
- **Navigation**: React Navigation v6
- **UI Library**: Native Base for consistent design
- **Platform Support**: iOS 14+ and Android API 24+

## Branch Strategy
1. Create feature branches: `mobile-[feature-name]`
2. Use conventional commits: `feat:`, `fix:`, `refactor:`, etc.
3. Test on both iOS and Android before committing
4. Run `npm run lint` and `npm run type-check` before commits

## Code Organization
- **Screens**: `/src/screens/[FeatureName]/[ScreenName]Screen.tsx`
- **Components**: `/src/components/[ComponentName]/index.tsx`
- **Navigation**: `/src/navigation/[NavigatorName]Navigator.tsx`
- **Services**: `/src/services/[serviceName].ts`
- **Store**: `/src/store/slices/[featureName]Slice.ts`

## Mobile-Specific Standards
- Use React Native's built-in components before third-party libraries
- Implement proper error boundaries for crash prevention
- Use AsyncStorage for local data persistence
- Follow platform-specific design guidelines (iOS Human Interface, Material Design)
- Optimize images and use vector graphics when possible

## Testing Requirements
- Unit tests with Jest and React Native Testing Library
- E2E tests with Detox for critical user flows
- Test on physical devices before major releases
- Performance testing with Flipper profiling

## Performance Guidelines
- Use FlatList for large datasets, never ScrollView
- Implement proper memoization with React.memo and useMemo
- Lazy load heavy components and screens
- Monitor bundle size and use code splitting when necessary
```

Example 4: Enterprise Backend Service

```
# Project: Payment Processing Microservice

## Service Architecture
- **Framework**: Spring Boot 3.1 with Java 17
- **Database**: PostgreSQL with JPA/Hibernate
- **Security**: OAuth 2.0 with JWT tokens
- **Documentation**: OpenAPI 3.0 specifications
- **Deployment**: Docker containers with Kubernetes

## Development Workflow
1. Create feature branch: `backend-[ticket-number]-[description]`
2. Write integration tests for all new endpoints
3. Update OpenAPI documentation for API changes
4. Run full test suite: `./gradlew test integrationTest`
5. Ensure Docker build succeeds: `docker build -t payment-service .`
6. Commit with ticket reference: `feat: [TICKET-123] Add payment validation`

## Code Standards
- Follow Google Java Style Guide
- Use Spring Boot conventions for package structure
- All REST endpoints must have comprehensive validation
- Use DTOs for all external API contracts
- Implement proper exception handling with @ControllerAdvice

## Package Structure
- `controller`: REST endpoints and request/response handling
- `service`: Business logic and transaction management
- `repository`: Data access layer with JPA repositories
- `config`: Configuration classes and beans
- `dto`: Data Transfer Objects for API contracts
- `entity`: JPA entities for database mapping
- `exception`: Custom exceptions and error handling

## Security Requirements
- All endpoints require authentication except health checks
- Validate all inputs with Bean Validation annotations
- Use parameterized queries to prevent SQL injection
- Log security events for audit trail
- Never expose internal entity models in API responses

```

Example 5: DevOps Infrastructure Project

```
# Project: Multi-Cloud Infrastructure Platform

## Infrastructure as Code Standards
- **Primary Tool**: Terraform with HCL
- **Cloud Providers**: AWS, Azure, GCP
- **Configuration Management**: Ansible playbooks
- **Container Orchestration**: Kubernetes with Helm charts
- **Monitoring**: Prometheus, Grafana, ELK stack

## Workflow Requirements
1. Create infrastructure branch: `infra-[environment]-[component]`
2. Run `terraform plan` and review changes carefully
3. Test in development environment first
4. Update documentation in `/docs/runbooks`
5. Peer review required for production changes
6. Use conventional commits with clear impact description

## Directory Structure
- `/terraform/[provider]/[environment]/`: Environment-specific configurations
- `/ansible/playbooks/`: Configuration management scripts
- `/k8s/[namespace]/`: Kubernetes manifests and Helm charts
- `/scripts/`: Automation and deployment scripts
- `/docs/`: Architecture decisions and runbooks

## Security and Compliance
- All secrets must use external secret management (AWS Secrets Manager, etc.)
- Enable encryption at rest and in transit for all data stores
- Implement least-privilege access policies
- Use service accounts, never personal credentials in automation
- Maintain audit logs for all infrastructure changes

## Deployment Principles
- Use blue-green deployments for zero-downtime updates
- Implement automatic rollback on health check failures
- Tag all resources with environment, owner, and cost-center
- Use infrastructure modules for reusability across environments
- Implement proper backup and disaster recovery procedures

## Monitoring and Alerting
- Set up alerts for resource utilization > 80%
```

Example 6: Open Source Library Development

```
# Project: DataValidator - Python Data Validation Library

## Library Design Principles
- **API Design**: Simple, intuitive, and Pythonic
- **Dependencies**: Minimal external dependencies
- **Compatibility**: Python 3.8+ support
- **Performance**: Optimize for speed and memory efficiency
- **Documentation**: Comprehensive with examples

## Development Workflow
1. Create feature branch: `feature-[functionality]` or `fix-[issue-number]`
2. Write tests first (TDD approach)
3. Ensure 100% test coverage for new code
4. Update documentation and examples
5. Run full test suite across Python versions: `tox`
6. Update CHANGELOG.md with clear user-facing description

## Code Quality Standards
- Follow PEP 8 with line length limit of 88 characters
- Use type hints for all public APIs
- Docstrings required for all public functions/classes (Google style)
- Use dataclasses or Pydantic for structured data
- Implement proper error handling with custom exceptions

## Project Structure
- `/src/datavalidator/`: Main library code
- `/tests/`: Comprehensive test suite
- `/docs/`: Sphinx documentation
- `/examples/`: Usage examples and tutorials
- `/benchmarks/`: Performance testing scripts

## Testing Requirements
- Unit tests with pytest for all functionality
- Property-based testing with Hypothesis for edge cases
- Performance benchmarks for critical paths
- Integration tests with popular data libraries (pandas, numpy)
- Test matrix: Python 3.8, 3.9, 3.10, 3.11, 3.12

## Documentation Standards
- README with quick start guide and installation
```

Best Practices Summary

Do's:

- ✅ Use clear, actionable language
- ✅ Provide specific tools and frameworks
- ✅ Define quality gates and testing requirements
- ✅ Include workflow processes
- ✅ Specify naming conventions
- ✅ Reference established design principles

Don'ts:

- ❌ Include task-specific details that only apply sometimes
- ❌ Write 1000-page documentation dumps
- ❌ Use vague instructions like "write good code"
- ❌ Forget to specify version control workflows
- ❌ Micromanage implementation details
- ❌ Include temporary or changing information

Remember the Balance

Claude.md gives Claude Code a specific target to hit while still allowing creative problem-solving within your constraints. It's your team's institutional knowledge distilled into essential, persistent context that makes every interaction more effective.

The goal is to scale your AI development labor by providing the right context at the right level - enough specificity to get what you want, but not so much constraint that it can't innovate within your boundaries.

Mark as completed

Like Dislike Report an issue

Go to next item

coach

Hi, Venkatesh!
How can I help?

Explain this topic in simple terms
Give me a summary
Give me practice questions
Give me real-life examples

Ask me anything